

Extending context

From 4K to a million tokens: positions, schedules, data, systems — and the difference between claimed and effective context

The product team wants "a full codebase in the prompt." Legal wants entire contracts. Someone forwards a competitor's announcement: *one million tokens of context*. Your model was pre-trained at 4,096. Before you promise anything, run three numbers. At 1M tokens, a GQA-8 model with 80 layers carries a KV cache of **328 GB — for one sequence** — four H100s' worth of memory before a single weight is loaded. The attention FLOPs to prefill that one sequence are $\sim 2.6 \times 10^{18}$ — about 44 minutes of a single H100's entire peak throughput, for one prompt. And the pointer you'd wave at all this, "just fine-tune on longer documents," fails before the money runs out: feed your 4K-trained model a 30K document and the loss doesn't degrade politely — it detonates, because the model has never seen the positions themselves. This chapter is about doing the extension properly: what breaks, in what order, and the now-standard recipe by which 4K models become honest 128K models and, with more machinery, nominal 1M ones.

One framing before the mechanics, because it will organize everything: **long context is not a feature you add — it is a stack you build**, and each layer of the stack fails independently. The positions must stretch (§21.2). The model must be *trained* at length, progressively, on data that actually exercises length (§21.3–21.4). The attention must be affordable at length — that was Chapter 13's job, and this chapter leans on it. The *systems* must physically fit a million-token batch (§21.5). And the result must be measured honestly, because the gap between advertised and effective context is the dirtiest open secret in the field (§21.7). Miss any layer and you ship a context window that is a lie in one of five different ways.

THE FIVE-LAYER STACK — each layer fails on its own

5 • Evaluation effective vs. advertised · §21.7	fails as: perfect needle demos, useless real documents RULER, multi-fact tasks — never NIAH alone
4 • Systems fitting the sequence at all · §21.5	fails as: OOM at training time — one sequence outgrows a node ring attention / context parallelism (with Ch. 14)
3 • Attention mechanism making length affordable · Ch. 13	fails as: it works, but serving it bankrupts you GQA/MLA cache, windows, indexer sparsity, hybrids
2 • Training recipe + data progressive stages · §21.3–21.4	fails as: window accepted, never used — or short-context skills regress staged lengths, length-mixed data, synthetic dependencies
1 • Positions RoPE θ scaling, YaRN · §21.2	fails as: loss detonates past the trained length — instantly stretch wavelenghts before you train a single long token

Figure 21.1 — Long context as a stack. Read bottom-up in build order: positions first (they gate everything), then the training recipe and its data, then affordable attention (Chapter 13's whole subject), then the parallelism to fit the sequences, and finally honest measurement. The chapter walks the same order.

21.1 Why long context is expensive — and why it breaks before it is expensive

Chapter 13 priced the steady-state costs, so here they only need their long-context values. Prefill compute grows quadratically: the $4L^2d$ FLOPs per layer that were a footnote at 4K become, at 1M, that 44-minutes-of-an-H100 number from the opening — which is why every 1M system prefills through sparse or chunked attention, never dense. Cache memory grows linearly but from a brutal base: 328 GB per million-token sequence for an 80-layer GQA-8 config; MLA's ~ 70 KB/token brings the same million to ~ 70 GB, still half a GPU (Chapter 13, §13.3). Decode bandwidth follows cache size, as always. These costs are real but *manageable* — the entire point of Chapter 13 — and none of them is the reason your 4K model fails at 30K.

It fails because of positions. RoPE (Chapter 3) encodes position by rotating each pair of dimensions at its own frequency, and the model has learned to read those rotation angles the way you read a clock. Feed it position 30,000 when it trained to 4,096 and the fast-rotating dimensions present angles the model has seen thousands of times — but the slow dimensions, the ones that encode *coarse* position, present configurations that occurred zero times in training. The model isn't interpolating on unfamiliar data; it is being asked to read a clock face that was never manufactured. Perplexity doesn't drift — it explodes. That is why positions are layer 1 of the stack: until they stretch, no amount of long data or clever attention even gets to run.

21.2 RoPE scaling: stretching the clock

Recall the machinery. Dimension pair i of a head rotates at angular frequency $\theta_i = b^{-2i/d}$ with base b (10,000 in the original recipe, $d = 128$ per head), which gives each pair a **wavelength** — the number of positions for one full rotation:

$$\lambda_i = \frac{2\pi}{\theta_i} = 2\pi b^{2i/d}$$

The pairs form a geometric ladder of clocks: the fastest completes a turn every ~ 6.3 tokens (local order), and at $b = 10,000$ the slowest takes $\approx 54,400$ positions per turn. Now the whole extension problem in one sentence: *a model trained to 4K has watched the fast clocks spin thousands of times but has seen only the first 7% of the slowest clock's dial* — everything past that is unmanufactured clock face. The fixes all amount to remapping angles so that inference-time positions land on dial regions the model has actually seen, and they escalate:

Position interpolation (PI) is the blunt version: divide every position by the scale factor $s = L_{\text{new}}/L_{\text{train}}$, squeezing the new range into the trained one.¹ It works, after a little fine-tuning, but it pays with resolution everywhere: at $s = 32$, adjacent tokens land 0.03 trained-position units apart, and the fast clocks — the ones encoding *local* order, which was fine and didn't need touching — get compressed too. Short-context quality visibly suffers.

Base scaling ("ABF") raises b instead: at $b = 500,000$ the slowest wavelength stretches to $\approx 2.56\text{M}$ positions; at $b = 10\text{M}$, to $\approx 49\text{M}$. This is the workhorse — Llama 3.1 ships at $b = 500\text{K}$, Qwen's 1M models push toward 10^7 — because it stretches the slow clocks (which needed it) far more than the fast ones (which didn't).²³

YaRN makes the per-dimension logic explicit: classify each pair by how its wavelength compares to the trained length.⁴ Pairs whose wavelength is much shorter than L_{train} rotated fully many times — leave them untouched (extrapolation is safe on a clock you've seen every angle of). Pairs whose wavelength exceeds L_{train} never completed a turn — interpolate those fully. Ramp smoothly in between, and add one scalar correction: soften the attention logits by a temperature factor $t = 0.1\ln s + 1$ (≈ 1.14 at $s=4$, ≈ 1.35 at $s=32$), because longer contexts feed more entries into every softmax and the score distribution

flattens. YaRN reaches usable long context with a fraction of the fine-tuning tokens PI needs, and it is what a `rope_scaling` config block in a modern checkpoint usually means:

```
# stretching a 4K-trained model toward 128K (transformers config idiom)
cfg.rope_theta = 500_000          # ABF: slowest wavelength 54K -> 2.56M positions
cfg.rope_scaling = {              # plus YaRN for the still-uncovered dims
    "rope_type": "yarn",
    "factor": 32.0,               # s = L_new / L_train
    "original_max_position_embeddings": 4096,
}
# sanity check the ladder before training a single token:
lam = lambda i, b, d=128: 2*math.pi * b**(2*i/d)
print(f"slowest pair: {lam(63, cfg.rope_theta):,.0f} positions/turn") # must exceed
L_new
```

the wavelength ladder (log scale) — 64 clocks per head, from fast to slow

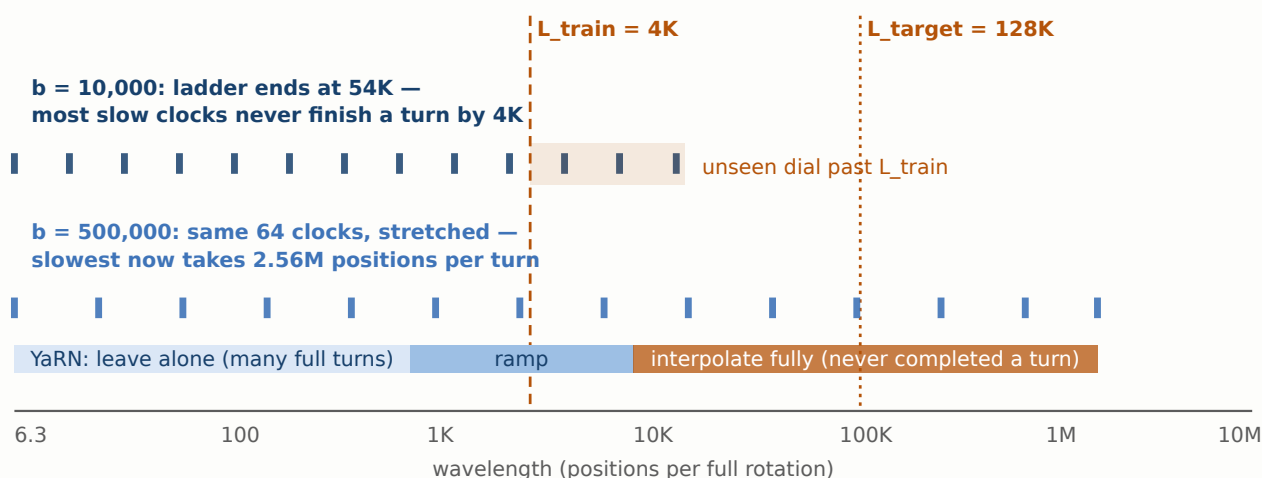


Figure 21.2 — Why base scaling works and what YaRN adds. Each tick is one dimension pair's wavelength on a log axis. At $b=10K$ the ladder tops out at 54K positions — and everything to the right of the 4K line is dial the model never saw. Raising b to 500K stretches the ladder to 2.56M; YaRN then treats the three zones differently: fast clocks untouched, slow clocks interpolated, a smooth ramp between.

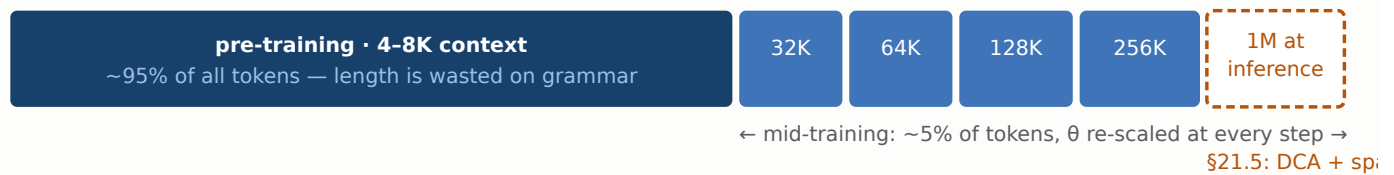
Whichever scheme you choose, one principle governs: **scaling positions is a permission slip, not an education**. After θ -scaling, the model can *represent* position 500,000 without detonating — it still has no idea what to *do* with a dependency that long, because it has never trained on one. That education is the next two sections, and skipping it produces the most common failure in the wild: a model that accepts a huge prompt and quietly uses the last few thousand tokens of it.

21.3 Progressive length training

Nobody trains at the target length from the start, for a reason Chapter 6's arithmetic makes obvious: attention FLOPs per token grow with the sequence around it, so a token learned inside a 256K window costs far more than the same token at 8K — and for the first trillions of tokens the model is learning grammar, facts, and code, none of which needs a quarter-million-token horizon. So the standard recipe is **staged**: pre-train at short length where tokens are cheap, then, in mid-training — exactly where this Part of the book lives — walk the context up in steps, re-scaling θ at each step and continuing training on

progressively longer data. Llama 3.1 went to its final 128K in **six stages**, spending roughly 800B tokens on the long-context phase — about 5% of its total budget — and reports that going gradually was what kept short-context evals from regressing.² Qwen's 1M-class models climb 4K → 32K → 64K → 128K → 256K with the base frequency raised alongside, and reach the final million *at inference time* on top of a 256K-trained model (§21.5).³

THE STAGED RECIPE (Qwen-style; Llama 3.1 uses six finer steps)



At every stage, three dials move together:

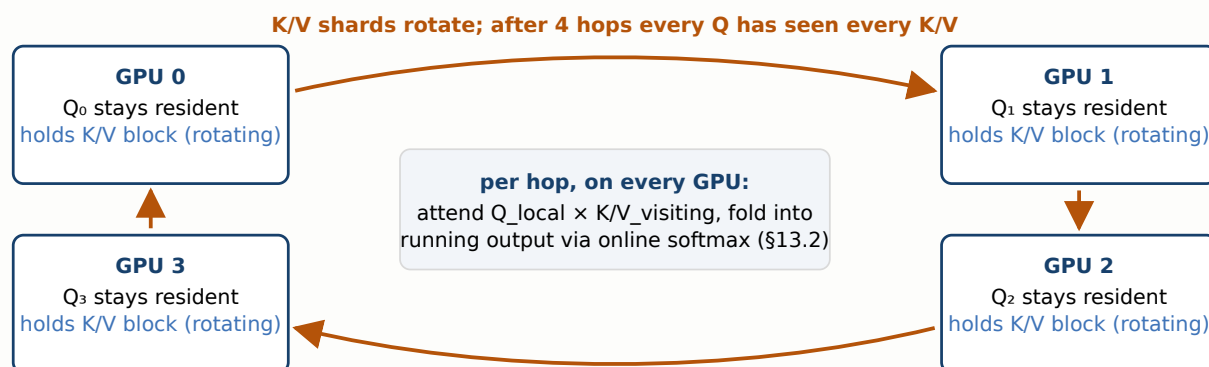
- θ (base frequency)** raised so the slowest wavelength stays ahead of the new length (Fig. 21.2)
- batch shape** tokens per batch held ~constant: sequences 8× longer ⇒ 8× fewer per batch (Ch. 17's stability rules st
- data mix** long documents upsampled but never exclusive — short data stays in the mix to protect short-context

Skipping steps is a false economy: each stage costs little (5% of budget total) and prevents the regression you'd pay weeks to debu

Figure 21.3 — Cheap tokens first, long tokens last. The overwhelming majority of training happens where tokens are cheap; the context ladder is climbed in a short, deliberate mid-training phase with θ , batch shape, and data mix moving together at each rung — and the final 4× beyond the last trained length is handled at inference (§21.5), not with more training.

Two mechanical notes that save real debugging time. **Batch shape:** optimizer stability (Chapter 17) cares about *tokens* per batch, so as sequences lengthen you cut the sequence count to hold token count roughly fixed — and your padding-free packing from Chapter 16 becomes load-bearing here, because long-document corpora are wildly length-skewed. **Where the sequences physically fit:** a 256K–1M sequence's activations don't fit one GPU, so the sequence itself is sharded across devices — **context parallelism**. Ring attention is the canonical form: each GPU holds one shard of Q, K, V; the K/V shards rotate around the ring while every GPU's queries accumulate attention against each passing block using the same online-softmax trick as FlashAttention (§13.2), communication overlapped with compute so the ring is nearly free when the per-shard tile is large enough.⁵ Exactness is preserved; only the memory is distributed. Chapter 14 covers the composition with the other parallelisms; the thing to remember from this side is that context parallelism is a *training-time* tool — at serving time you'd rather not pay a ring per request, which is one more reason the serving path leans on §21.5's tricks instead.

RING ATTENTION — one 1M sequence across 4 GPUs (250K tokens each)



Communication hides behind compute when block-tiles are big enough — FlashAttention's exactness argument, stretched across a

Figure 21.4 — Ring attention: FlashAttention across machines. The sequence is sharded; queries never move; K/V blocks make one lap of the ring while each GPU folds every visiting block into its running softmax. Attention stays mathematically exact — only the memory and the traffic are distributed.

21.4 Long-context data: length is necessary, dependency is the point

The naive data plan — "filter the corpus for long documents, train on those" — fails in an instructive way: most naturally long documents are only *locally* coherent. A concatenated novel teaches almost nothing about using position 200,000, because predicting its next token rarely requires anything from page one. What trains long-range ability is **dependency at distance**, and the field's recipes manufacture it deliberately. Qwen's long-context stages mix upsampled natural long documents (books, code repositories, multi-file projects — code is the best natural source of true long dependencies) with *synthetic* tasks built for reach: fill-in-the-middle at document scale, keyword and passage retrieval across the full window, paragraph reordering that cannot be solved locally.³ The synthetic fraction matters more than its volume: it converts "long input" into "long supervision."

The second rule of the mix protects what you already built: **short data never leaves**. A mid-training phase run purely on long sequences measurably regresses short-context quality — the everyday behavior 95% of your traffic exercises — and the standard defense is keeping a substantial share of ordinary-length data in every long stage (Llama 3.1's gradual staging exists precisely to manage this balance²). Treat the long-context phase like Chapter 20's annealing: a deliberate change of diet, with regression evals (Chapter 9) watching the old capabilities while the new one grows. And packing discipline from Chapter 16 applies with a twist — at 256K you are often packing *few* real documents per sequence with cross-document attention masked, so the batch actually contains the long documents you paid to curate rather than a mosaic of short ones wearing a long sequence length.

21.5 Serving past what you trained: the last 4× is free-ish

Every stage of the ladder costs training compute — so the labs stop the ladder early and cover the final stretch with inference-time machinery. The workhorse is **Dual Chunk Attention** (DCA), and its idea is disarmingly simple: the model trusts relative positions up to its trained length, so never show it a bigger one.⁶ Cut the long sequence into chunks smaller than the trained window and compute attention with three reused position patterns — ordinary positions *within* a chunk, a fixed compressed pattern *between* distant chunks, and a careful pattern for neighboring chunks so locality stays exact. Every relative distance the model ever sees now lands inside the range it trained on; nothing is fine-tuned, nothing is learned. Stacked on a 256K-trained model, this is precisely how Qwen serves 1M: DCA for positions, plus sparse prefill (MInference-style dynamic patterns⁷ — a

cousin of §13.5's indexer family) to make the million-token prompt computable, for a reported 3–7× prefill speedup at accuracy parity with full attention.³

DCA — never show the model a distance it wasn't trained on



within chunk 3: real relative positions, 0...256K — exactly what training saw

to neighbor (chunk 2): successor pattern keeps token-adjacent distances exact across the seam

to distant chunks (0, 1): one compressed "far away" position pattern, reused for all of them — the model needs "long ago," not "position 793,214"

Zero training. Stacked with sparse prefill (§13.5's indexer family), this is the standard 256K-trained → 1M-served bridge.

Figure 21.5 — Dual Chunk Attention. Three reused position patterns — intra-chunk, successor, inter-chunk — guarantee every relative distance the model computes falls inside its trained range. The last 4× of advertised context typically costs no training at all.

One design from the frontier deserves a note because it moves this logic *into* the architecture: Llama 4's **iRoPE** interleaves layer types — most layers use RoPE with chunked local attention, while periodic global-attention layers carry *no positional encoding at all* (NoPE), plus an inference-time temperature scaling on attention scores. The bet is that position-free global layers cannot have a positional out-of-range problem, so length generalizes further — it is the architectural echo of DCA's "distant means far away, not a number" — and it is how the 10M-token claim on Llama 4 Scout is constructed. Treat the number itself with §21.7's skepticism; the design idea is worth knowing regardless.⁸

21.6 Compressive memory: the road not (yet) taken

CONCEPT BOX · INFINI-ATTENTION

There is a third road to unbounded context, and you have already met both of its halves. Google's **Infini-attention** processes the stream in segments: within a segment, ordinary softmax attention; everything older is folded into a **compressive memory** M that is precisely §13.5's linear-attention state — written with the same delta-rule update, read associatively by each query — and a learned per-head gate β blends the two views:⁹

$$A = \underbrace{\sigma(\beta) \frac{\phi(Q) M_{s-1}}{\phi(Q) z_{s-1}}}_{\text{read the infinite past}} + (1 - \sigma(\beta)) \underbrace{\text{Attn}(Q, K, V)}_{\text{exact local attention}}, \quad M_s = M_{s-1} + \phi(K)^\top (V - \text{retrieved})$$

Fixed memory per head regardless of length — the paper reports 114× memory compression against a cache, passkey retrieval at 1M after light fine-tuning, and it is the intellectually cleanest unification in this whole area: softmax attention for the recent, recurrent state for the ancient, learned arbitration between them.

Why it gets a box and not a section: two years on, no shipped flagship uses it. Public reproductions have struggled to match the reported quality, and the labs that actually serve million-token products built them from the stack in Figures 21.1–21.5 instead. Teach it as the conceptual bridge between Chapter 13's two roads — and as a live reminder that in

this field, elegant and adopted are different properties. If the hybrid-stack trend (§13.5) continues, something with this shape may yet win; the delta-rule state it relies on is exactly what Gated DeltaNet layers already compute.

21.7 Measuring it: effective context ≠ advertised context

The demo that sells long context is needle-in-a-haystack: hide "The magic number is 7,481,392" at position 600,000, ask for it, and print the green heat-map when the model finds it. The demo is real and the demo is nearly meaningless: retrieving one verbatim string is the *easiest possible* use of a long window — closer to grep than to reading — and modern models ace it far beyond the lengths at which they stop being useful. When RULER replaced the single needle with tasks that require using the window (multiple needles, aggregation across the document, variable tracking, multi-hop chains), the picture inverted: of the models it tested claiming 32K or more, roughly half failed to hold their own baseline quality even *at* 32K, and models advertising 128K commonly showed effective lengths of 4–64K.¹⁰ Later "context rot" studies sharpened the point: performance degrades *with input length even on trivially simple tasks*, non-uniformly and per-model — the window is not a flat plateau but a slope.¹¹

Table 21.1 — Advertised vs. effective, illustrative readings from RULER-style evaluation.¹⁰ The exact numbers move with every release; the *gap* is the durable lesson. Measure your own model before quoting a window.

WHAT THE CARD SAYS	WHAT RULER-STYLE TESTING TYPICALLY FINDS	RULE OF THUMB
"128K context"	baseline quality held to ~32–64K on strong models; far less on weak ones	effective ≈ ¼–½ of advertised, if trained well
"1M context"	single-needle retrieval fine; multi-fact aggregation degrades far earlier	quote per-task lengths, not one number
"perfect NIAH to 10M"	says almost nothing about reading comprehension at 10M	NIAH is a smoke test, never the headline

The practical protocol, in one paragraph. Evaluate at *every* length on the ladder (8K, 32K, 128K, ...), not only the maximum, and plot quality-versus-length — the knee of that curve is your effective context, and the number you should put in front of the product team. Use a battery: RULER's task families for controlled stress, at least one realistic task from your own domain (a real contract, a real codebase question), and your standard short-context suite *rerun at every stage* of §21.3 to catch regression. Position matters too: test the same fact at the start, middle, and end of the window — degradation is rarely uniform, and "lost in the middle" failures hide from end-anchored tests. And define the deliverable of the whole chapter as that curve, not a context length: a window whose quality-versus-length curve you have measured and published internally is a feature; an advertised number is marketing.

21.8 What the big labs do

Table 21.2 — Long-context recipes at the frontier. Advertised windows, with the machinery behind them.

MODEL	WINDOW	POSITIONS	RECIPE
-------	--------	-----------	--------

MODEL	WINDOW	POSITIONS	RECIPE
Llama 3.1 ²	128K	RoPE $\theta = 500K$	Six-stage extension from 8K, ~800B long-context tokens (~5% of budget); gradualism explicitly credited for protecting short-context evals
Qwen2.5-1M ³	1M	ABF toward 10^7	Staged 4K $\rightarrow$ 256K training + synthetic dependency data; final 4 $\times$ at inference via DCA + MInference-style sparse prefill (3–7 $\times$ speedup)
Gemini 1.5/2.x ¹²	1–2M product	undisclosed	Near-perfect single-needle to 10M in research; methods unpublished — context-parallel training widely assumed
Llama 4 Scout ⁸	10M claimed	iRoPE: RoPE local + NoPE global layers	Architectural length generalization + inference temperature scaling; effective length at 10M debated — Table 21.1's lesson applies
MiniMax-Text-01 ¹³	1M train / 4M serve	hybrid	Lightning (linear) attention 7:1 over softmax layers — the Ch. 13 hybrid road taken to 4M
Gemma 3 ¹⁴	128K	global layers $\theta = 1M$, locals $\theta = 10K$	5:1 local:global stack (§13.4): only every sixth layer pays long-range positions and cache
DeepSeek-V3.2 ¹⁵	128K	MLA + decoupled RoPE	Chose cheap-at-128K (DSA top-k sparsity) over a bigger headline number — a deliberate product decision worth noticing

Read column-wise, the table repeats this chapter's thesis: *nobody* reaches a big window with one trick. Every row is positions + staged training + a Chapter 13 mechanism + systems — and the two most honest rows are instructive opposites: Qwen documents the full recipe including where training stops and inference tricks take over, while DeepSeek declines the 1M arms race entirely and spends the effort making 128K nearly free. Both are coherent strategies; the incoherent one is advertising a number the stack doesn't support.

21.9 Practical considerations

Check the ladder before every stage. Two minutes of arithmetic — is the slowest wavelength comfortably ahead of the new target length? is the YaRN ramp covering the never-rotated dims? — prevents the classic silent failure where stage three trains at 128K against positions configured for 64K, producing a model that "trained fine" and evaluates like a coin flip past the boundary. Put the wavelength check from §21.2's code into the launch script, not the wiki.

Long context multiplies every other bill — anticipate the interactions. Activation memory per sequence grows linearly (Chapter 15's checkpointing goes from useful to mandatory past ~64K); attention's share of step time grows toward dominance (the $L^* \approx 6d$ crossover of Chapter 13 sits mid-ladder, so profile per stage — the optimal recompute/parallelism config at 8K is wrong at 256K); and evaluation cost explodes quietly (one 1M-token eval prompt costs as much prefill as ~250 4K prompts — budget eval compute per stage or you will silently stop measuring).

The serving story is a different story. Everything in §21.3 (ring attention, context parallelism) is for training. At serving time the levers are Chapter 13's cache reducers, chunked prefill, prefix caching for shared long prompts, and DCA/sparse prefill — and the honest question from Chapter 8's economics: for your actual traffic, is a 256K window with retrieval feeding it a better product than a native 1M window? For most deployments the answer is yes for years — long context and retrieval are complements, not rivals, and the cheapest million tokens are the 990,000 you didn't put in the prompt.

Common pitfalls

Scaling θ and calling it done.

The permission-slip fallacy of §21.2: no long training data means the window is accepted, not used. NIAH will even pass. RULER won't.

Training only on long sequences in the extension phase.

Short-context regression is the most common casualty; keep short data in every stage and rerun the short suite every stage.

Long inputs, short supervision.

If every training target depends only on nearby tokens, length teaches nothing — manufacture distant dependencies (§21.4) or the model learns to ignore exactly what you're paying for.

Packing mosaics that impersonate long documents.

A 256K sequence of 64 packed short docs (Ch. 16) trains zero long-range behavior. Audit what's actually inside the long batches.

Quoting the advertised number.

Effective context is a per-task curve (§21.7). Internally, always the curve; externally, the knee of the curve.

Evaluating only at the end anchor.

"Lost in the middle" failures are invisible when your probe always sits at the end of the prompt. Sweep positions.

21.10 Decision framework

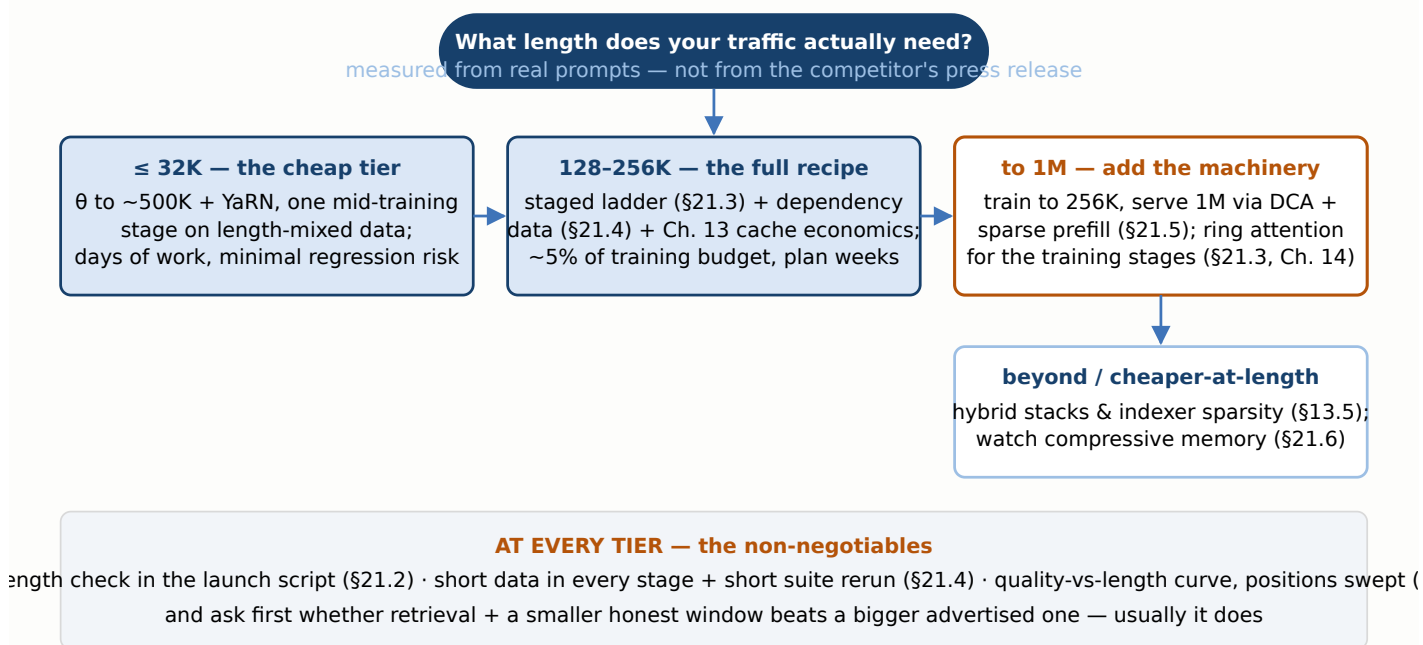


Figure 21.6 — Choosing a context strategy. Tiers escalate left to right, and each tier includes everything to its left. The row at the bottom is not optional at any tier — it is the difference between a measured capability and an advertised number.

Cost-benefit, bluntly. The 32K tier is nearly free and nearly always worth it — θ -scaling plus one stage buys the level where most real traffic saturates. The 128–256K tier costs ~5% of a training budget and real engineering attention (packing, parallelism, per-stage evals) — pay it when your product actually reads documents of that size, not for the spec sheet. The 1M tier's training cost is modest *given* the 256K base (DCA is free; sparse prefill is an inference-stack project), but its serving economics are unforgiving — Chapter 13's cache table decides whether you can afford your own success. **How this scales:** the ladder's cost is dominated by its top rung (quadratic FLOPs live there), which is why every recipe is geometric in its stage lengths; and the payoff structure argues for lagging the frontier deliberately — effective-context quality at 128K today beats advertised 1M at the frontier's price, unless long documents *are* your product.

21.11 Summary

Long context is a five-layer stack, and each layer fails alone. Positions gate everything: RoPE's dimension pairs are clocks on a geometric ladder of wavelengths, a model trained at 4K has seen only a sliver of the slow dials, and extension begins by stretching them — base scaling to move the ladder (54K $\rightarrow$ 2.56M positions at $\theta = 500K$), YaRN to treat fast, slow, and in-between dims each correctly, plus its $0.1 \ln s + 1$ attention temperature. But stretched positions are a permission slip, not an education: the capability comes from progressive length training — short pre-training for the cheap 95%, then a staged mid-training ladder (θ , batch shape, and data mix moving together) on data whose defining property is *dependency at distance*, natural (code, books) and synthetic (document-scale FIM, cross-window retrieval), with short data retained at every stage to protect what you already built. Ring attention shards the sequence so the stages physically fit; DCA plus sparse prefill serves past the trained length for free, which is how 256K-trained models honestly ship 1M; Infini-attention's compressive memory remains the elegant unadopted bridge. And the deliverable is a curve, not a number: single-needle demos saturate long after real usefulness has decayed, RULER-style testing routinely finds effective context at a quarter to a half of advertised, and the

knee of your own quality-versus-length curve — swept across positions, per task, per stage — is the only context length worth telling anyone you have.

References

1. Chen, S., Wong, S., Chen, L., & Tian, Y. (2023). Extending Context Window of Large Language Models via Positional Interpolation. arXiv:2306.15595.
2. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. RoPE $\theta = 500K$; six-stage context extension to 128K over $\sim 800B$ tokens.
3. Yang, A., et al. (2025). Qwen2.5-1M Technical Report. arXiv:2501.15383. Progressive 4K $\rightarrow$ 256K training with adaptive base frequency; synthetic long-dependency data (FIM, retrieval, reordering); 1M at inference via DCA + sparse prefill with 3–7 $\times$ speedups.
4. Peng, B., Quesnelle, J., Fan, H., & Shippole, E. (2024). YaRN: Efficient Context Window Extension of Large Language Models. *ICLR*. arXiv:2309.00071. Wavelength-aware interpolation and the ln-scale attention temperature.
5. Liu, H., Zaharia, M., & Abbeel, P. (2023). Ring Attention with Blockwise Transformers for Near-Infinite Context. arXiv:2310.01889.
6. An, C., et al. (2024). Training-Free Long-Context Scaling of Large Language Models (Dual Chunk Attention). *ICML*. arXiv:2402.17463.
7. Jiang, H., et al. (2024). MInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention. *NeurIPS*. arXiv:2407.02490.
8. Meta AI (2025). The Llama 4 Herd. Meta AI blog, April 2025. iRoPE (interleaved NoPE global + RoPE local layers), inference temperature scaling, 10M-token claim on Scout.
9. Munkhdalai, T., Faruqui, M., & Gopal, S. (2024). Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. arXiv:2404.07143. Compressive memory + local attention with a learned gate; 114 $\times$ memory compression, 1M passkey.
10. Hsieh, C.-P., et al. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? *COLM*. arXiv:2404.06654. Multi-needle, aggregation, tracing and QA task families; effective context routinely far below advertised.
11. Hong, K., Levin, A., & Cook, J. (2025). Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma technical report. Performance declines with input length even on simple tasks, non-uniformly across models.
12. Gemini Team, Google (2024). Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context. arXiv:2403.05530.
13. MiniMax (2025). MiniMax-01: Scaling Foundation Models with Lightning Attention. arXiv:2501.08313. Hybrid linear:softmax 7:1; 1M training window, 4M inference.
14. Gemma Team, Google DeepMind (2025). Gemma 3 Technical Report. arXiv:2503.19786. 5:1 local:global with $\theta = 1M$ on global layers, 10K on locals.
15. DeepSeek-AI (2025). DeepSeek-V3.2-Exp: Boosting Long-Context Efficiency with DeepSeek Sparse Attention. Technical report. 128K with top-k sparse reads over MLA (see §13.5).

Related: *Chapter 13 — Making attention cheap* (the mechanism layer of this chapter's stack) · *Chapter 14 — Distributing the model* (where ring attention composes with the other parallelisms) · *Chapter 16 — Sequence packing* (what's really inside a 256K batch) · *Chapter 29 — the case study*. · [Training Efficient LLMs](#)